



SWIFT BÁSICO

Instructores:

Cortés Alvarado Iván Daniel

Martinez Ortiz Marcos Uriel

Rojas Pech Eric

IOS DEVELOPMENT LAB

CONTACTO

Correo para entregables:
cursoswiftbasico@gmail.com

Drive:
https://drive.google.com/drive/folders/1-4J1NI-PQSIZE2pJ99IOKz6mb-48jsDb?usp=share_link

Grupo de WhatsApp:
<https://chat.whatsapp.com/DffvtQeZSjm5anPfCyJqzj>



EVALUACIÓN:

Tareas (30%)

Google forms

Ejercicios (70%)

Playgrounds

Total: 100%

*Para obtener diploma es necesario obtener como mínimo un 80%.

Fecha límite de entrega: 1 de febrero de 2023



TEMARIO DEL CURSO

01. Variables y constantes

- 1.1 Constantes
- 1.2 Variables
- 1.3 ¿Constante o variable?
- 1.4 Nombrar constantes y variables
- 1.5 Comentarios

02 Tipos de datos

- 2.1 Tipos
- 2.2 Seguridad de tipo
- 2.3 Inferencia de tipo
- 2.4 Valores requeridos

TEMARIO DEL CURSO

03. Operadores

- 3.1 Asignar un valor
- 3.2 Aritmética básica
- 3.3 Asignación compuesta
- 3.4 Operador de resto
- 3.5 Orden de las operaciones
- 3.6 conversión de tipo numérico

04. Cadenas y caracteres

- 4.1 Concatenación e interpolación
- 4.2 Equivalencia y comparación de cadenas
- 4.3 Temas de cadenas más avanzados
- 4.4 Unicode

TEMARIO DEL CURSO

05. Colecciones

- 5.1 Arreglos
- 5.2 Tipos de arreglos
- 5.3 Trabajar con arreglos
- 5.4 Diccionesarios
- 5.5 Agregar/eliminar/modificar un diccionario
- 5.6 Acceder a un diccionario

TEMARIO DEL CURSO

06. Estructuras de control

- 6.1 Operadores lógicos y de comparación
- 6.2 Instrucciones "IF"
- 6.3 Instrucciones "IF-ELSE"
- 6.4 Valores Booleanos
- 6.5 Instrucción "Switch"
- 6.6 Operador Condicional Ternario
- 6.7 Ciclos "For"
- 6.8 Ciclos "While"
- 6.9 Ciclos "Repeat-While"
- 6.10 Instrucciones de transferencias de control

TEMARIO DEL CURSO

07. Funciones

- 7.1 Definir una función
- 7.2 Parametros
- 7.3 Etiquetas de argumento
- 7.4 Valores de parámetros predeterminados
- 7.5 Valores de devolución

08. Closures (cierres)

- 8.1 Sintaxis
- 8.2 Pasar closures como argumentos
- 8.3 Devolución de valores en closures
- 8.4 Closures como parámetros

TEMARIO DEL CURSO

09. Enumeraciones (enums)

- 9.1 Flujo de control
- 9.2 Ventajas de la seguridad de tipo

10. Estructuras

- 10.1 Instancias
- 10.2 Inicializadores
- 10.3 Métodos de instancia
- 10.4 Métodos mutantes
- 10.5 propiedades calculadas
- 10.6 Observadores de propiedad
- 10.7 Propiedades y métodos de tipo
- 10.8 Copia
- 10.9 Self

TEMARIO DEL CURSO

11. Clases

- 11.1 Herencia
- 11.2 Referencias
- 11.3 Inicializadores de miembros
- 11.4 Clase o estructura

12. Opcionales

- 12.1 Nil (Nulo)
- 12.2 Especificar el tipo de un opcional
- 12.3 Trabajar con valores opcionales
- 12.4 Funciones y opcionales
- 12.5 Inicializadores falibles
- 12.6 Encadenamiento opcional
- 12.7 Opcionales extraídos de forma implícita

TEMARIO DEL CURSO

13. Guard

- 11.1 Guard con opcionales

14. Protocolos

- 14.1 Imprimir información con CustomStringConvertible
- 14.2 Comparar información con Equatable
- 14.3 Creando un protocolo
- 14.4 Delegación

15. extensiones

- 15.1 Añadir propiedades calculadas
- 15.2 Añadir métodos de instancia o tipo
- 15.3 Organización de código

01. VARIABLES Y CONSTANTES



- **1.1. Constantes**
- **1.2 Variables**
- **1.3 ¿Constante o variable?**
- **1.4 Nombrar constantes y variables**
- **1.5 Comentarios**

VARIABLES Y CONSTANTES INTRODUCCIÓN

En la programación en general se trata de trabajar con datos, en esta lección aprenderás como manejar y almacenar los datos, aprenderás a definir constantes para valores que no cambian y variables para valores que si lo hacen.

VARIABLES Y CONSTANTES

1.1 Constantes

Si quieres denominar un valor que no cambie durante la vida del programa, deberas usar una constante.

```
let name = "Jhonn"  
print(name)
```

Debido a que "name" es una constante, no puede modificar su valor y el compilador marca error

```
let name = "Jhonn"  
print(name)
```

```
name = "Miguel"
```



Cannot assign to value: 'name' is a 'let' constant

VARIABLES Y CONSTANTES

1.2 Variables

Si quieres denominar un valor que cambie durante la vida del programa, deberas usar una variable.

```
var age = 29  
print(age)
```

Debido a que "age" es una Variable, puede modificar su valor y el compilador no marca error

```
var age = 29  
print(age)  
age = 77  
print(age)
```



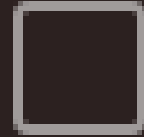
29
77

VARIABLES Y CONSTANTES

1.2 Variables

Puedes asignar constantes y variables de otras constantes y variables. Esta funcionalidad es útil si quieres copiar un valor a una o más variables

```
let puntajeInicial = 100
var jugador1 = puntajeInicial
var jugador2 = puntajeInicial
print(jugador1)
print(jugador2)
jugador1 = 200
print(jugador1)
```


100
100
200

VARIABLES Y CONSTANTES

1.3 ¿Constante o variable?

“Imagina que estás calculando información sobre la distancia recorrida en un viaje. El programa se puede reutilizar para rastrear muchos viajes a lo largo del tiempo, pero rastrea un viaje a la vez. ¿Cómo representarías los siguientes datos en tu código?”

VARIABLES Y CONSTANTES

1.3 ¿Constante o variable?

Ubicación de inicio: esta es una coordenada de GPS del lugar donde comenzó tu viaje. Una vez que comiences a rastrear un viaje en tu programa, la ubicación no cambiará. Representarás este valor con una constante.

Destino: esta coordenada de GPS es el lugar adonde quieres llegar. Tu app se puede usar para muchos destinos, por lo que podrías pensar que esto se representaría con una variable. Pero una vez que tu programa comience a rastrear un viaje, el destino no cambiará. Representarás este valor con una constante.

Ubicación actual: la coordenada de GPS de tu ubicación actual cambiará cada vez que te muevas. De modo que representarás este valor con una variable.

Distancia recorrida: ¿cuánto has viajado desde tu punto de partida? Este valor cambia a medida que te mueves. Lo representarás con otra variable.

Distancia restante: ¿cuánto debes viajar para llegar a tu destino? La distancia restante cambia a medida que te mueves. Representarás este valor con una variable.

VARIABLES Y CONSTANTES

1.3 ¿Constante o variable?

¿Por qué deberías usar constantes?

1. Refuerzas la seguridad por el compilador
2. Existen optimizaciones especiales que el compilador puede realizar para valores constantes. Si usas constantes para valores que no cambiarán, el compilador puede hacer suposiciones de bajo nivel sobre cómo almacenar el valor, de esta forma la app se ejecuta de manera mas rápida.
3. es la forma idiomática o aceptada de hacer cosas en Swift

VARIABLES Y CONSTANTES

1.4 Nombrar constantes y variables

Existen reglas para nombrar constantes o variables. El compilador impone las reglas, por lo que no podrás compilar ni ejecutar tu programa si las infringes

- Los nombres no pueden contener símbolos matemáticos.
- Los nombres no pueden contener espacios.
- Los nombres no pueden comenzar con un número

```
let π = 3.14159
let 一百 = 100
let 🎲 = 6
let mañana = "Tomorrow"
let anzahlDerBücher = 15 // numberOfBooks
```

VARIABLES Y CONSTANTES

1.4 Nombrar constantes y variables

Practicas recomendadas

1. Los nombres de las constantes y las variables deben ser claros y descriptivos, lo cual facilita la comprensión del código cuando vuelvas a consultarlo más tarde. Por ejemplo, **mejorPuntaje** es mejor que **n** y **personasCercanas** es mejor que **cerca**.
2. Para que los nombres sean claros y descriptivos, con frecuencia deberás usar varias palabras en un nombre. Si juntas dos o más palabras, la convención es usar camel case. Pon la primera letra del nombre en minúscula y luego pon la primera letra de cada palabra nueva en mayúscula.

Ejemplos:

- nombrehemano -----> nombreHermano
- cocherojo -----> cocheRojo
- casavacacioneseuropatiaana-----> casaVacacionesEuropaTiaAna

VARIABLES Y CONSTANTES

1.5 Comentarios

A medida que el código se hace más complejo, resulta muy útil dejar comentarios en partes específicas del código que te recuerden como funcionan. Esto resulta útil no solo para ti, también para tus compañeros que no saben la forma en la que codificas

```
//Si el chanchito murio de cancer el valor es 1
//Si murio de viejo 0.5
let deQueMurioElChanchito = 0.5

/*
  Bien, hoy te voy a contar la historia de como abandone
  todo mi futuro por mi ex....
  Una tarde de verano, mientras comia chilaquiles de
  donRata .....
*/
```

02. TIPOS DE DATOS



- **2.1 Tipos**
- **2.2 Seguridad de tipo**
- **2.3 Inferencia de tipo**
- **2.4 Valores requeridos**

TIPOS DE DATOS

2.1 Tipos

Cada constante o variable en Swift tiene un tipo que describe su tipo de valor. En los ejemplos que han visto, 29 es un número entero representado por el tipo de Swift `Int`. Del mismo modo, "John" es una cadena de caracteres representada por el tipo `String` (cadena) de Swift.

Nombre	Tipo	Próposito	Ejemplo
Integer	<i>Int</i>	Representa numeros enteros	4
Double	<i>Double</i>	Representa números que requerían puntos decimales, numeros reales	4.20
Boolean	<i>Bool</i>	Representa los valores true o false	true
String	<i>String</i>	Representa texto	"Hola Mundo!"

TIPOS DE DATOS

2.1 Tipos

Integer (Enteros):

Podremos almacenar números positivos y negativos sin decimales. El cero se considera número entero.

Palabra reservada: `Int`

Ejemplo:

```
let numberInt = 1  
let numberInt: Int = 0
```

Rango:

`..., -2, -1, 0, 1, 2, ...`

TIPOS DE DATOS

2.1 Tipos

Double (Decimales):

Podremos almacenar números positivos y negativos que requieran puntos decimales o números reales.

Palabra reservada: `Double`

Ejemplo:

```
var numberDouble = 3.15  
var numberDouble: Double = -0.4
```

Rango racionales:

..., -2.5, -1.5, 0.5,
1.5, 2.5, ...

Rango irracionales:

$\sqrt{2}$ e

TIPOS DE DATOS

2.1 Tipos

String (cadenas de texto):

Podremos almacenar varios caracteres (letras, signos y números que forman palabras o frases)

Palabra reservada: `String`

Ejemplo:

```
let nombre = "Fernanda Reyes"
```

```
var address: String = "Av. Universidad 3000, Coyoacan, CDMX"
```

TIPOS DE DATOS

2.1 Tipos

Booleanos:

Tipo de valor cuyos valores son True (verdadero) o False (falso)

Palabra reservada: `Bool`

Ejemplo:

```
let verdadero = true  
var falso: Bool = false
```

TIPOS DE DATOS

2.2 Seguridad de tipo

Los lenguajes de tipo seguro requieren que tengas claro los tipos de valores con los que puede funcionar tu código. Por ejemplo, si una variable de tu código espera un número entero (Int), no puedes pasarlo a un tipo Double o a uno String.

Ejemplo:

```
var score = 1_000_000_000
```

```
var playerName = "Julio"
```

```
score = playerName
```


TIPOS DE DATOS

2.3 Inferencia de tipo

Es opcional el especificar el tipo de valor cuando declaras una constante o una variable. Esto se llama inferencia de tipo. Swift usa las inferencias de tipo para hacer suposiciones sobre el tipo en función del valor asignado a la constante o la variable.

Ejemplo:

```
var score = 1_000_000_000 (Int)
var playerName = "Julio" (String)
var pi = 3.1415927 (Double)
```

```
var score: Int = 1_000_000_000
var playerName: String = "Julio"
var pi: Double = 3.141592
```

```
let number: Double = 3
```

Consola:

3.0

TIPOS DE DATOS

2.4 Valores requeridos

Siempre que se defina una constante o una variable, se debe especificar un tipo mediante una anotación de tipo o bien asignarle un valor que el compilador pueda usar para inferir el tipo.

No valido:

```
var X (Esto causaría un error)  
print(X) (Esto causaría un error)
```

Valido:

```
var X: Int  
X = 10  
print(X)
```

03. OPERADORES



- **3.1 Asignar un valor**
- **3.2 Aritmética básica**
- **3.3 Asignación compuesta**
- **3.4 Operador de resto**
- **3.5 Orden de las operaciones**
- **3.6 conversión de tipo numérico**

Un operador es un símbolo

En programación, un operador relaciona dos símbolos y hace que el código funcione al realizar una acción sobre el elemento que se encuentra a su izquierda y a su derecha.

3.1 Asignar un valor

Un operador siempre se encuentra en medio de dos elementos:
el de la izquierda y el de la derecha.

Al elemento de la izquierda se le asigna el valor de la derecha.

El valor asignado ya sea a una constante o variable puede ser numérico o no.



3.2 Aritmética básica

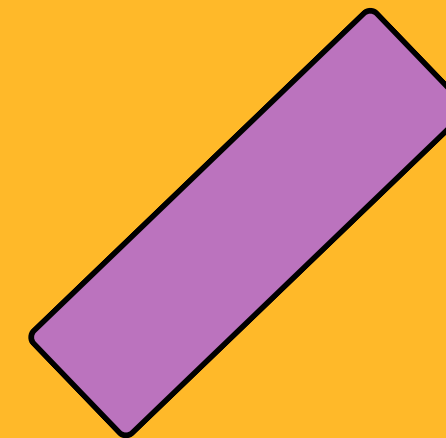
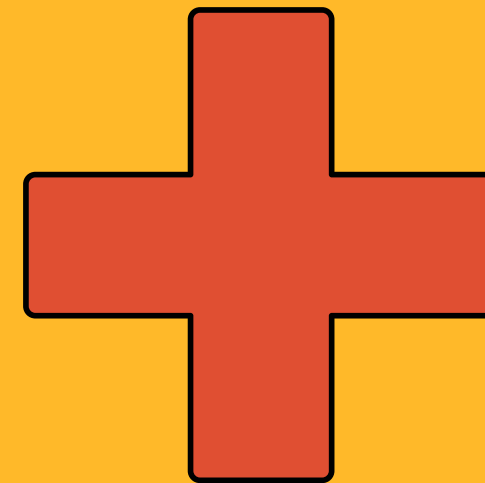
Las operaciones aritméticas básicas son:

suma (+),

resta (-),

multiplicación (*) y

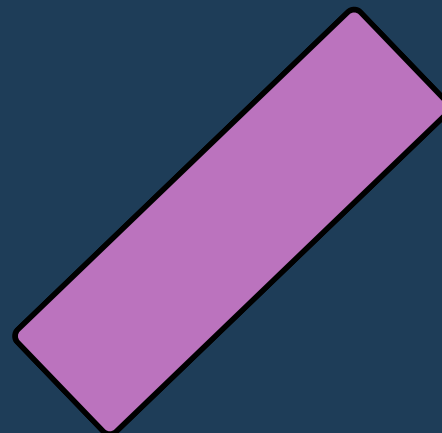
división (/).





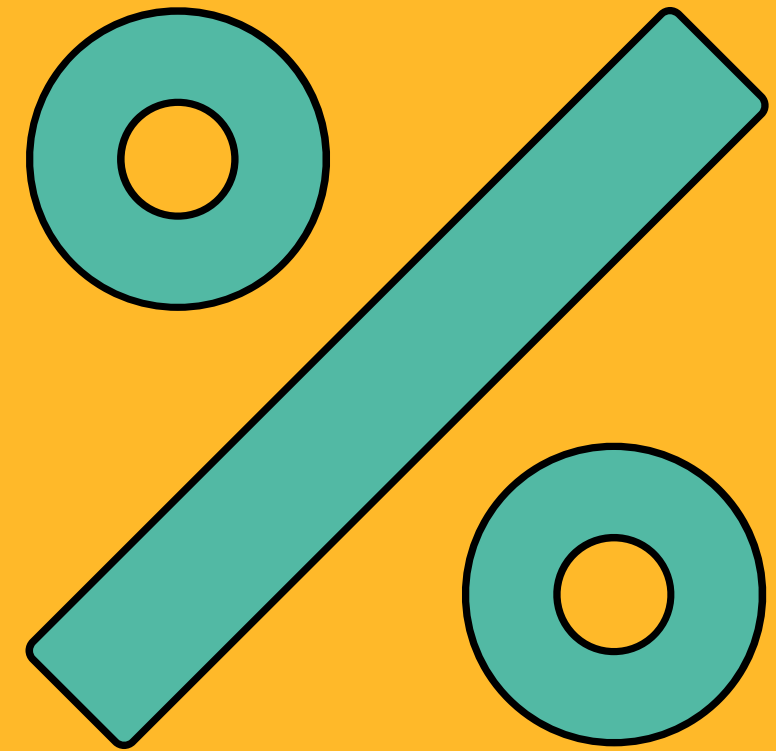
3.3 Asignación compuesta

La manera más sencilla de modificar un valor que ya ha sido asignado es utilizando un operador de asignación compuesta. Para utilizarlo es necesario escribir el operador de asignación (=) después de un operador aritmético.



3.4 Operador de resto

Se utiliza para calcular el resto de la división entre dos números enteros Int.



3.5 Orden de las operaciones

Se leen de izquierda a derecha. La multiplicación y la división tienen prioridad sobre la suma y la resta. Los paréntesis son quienes tienen mayor prioridad.

El operador resto tiene la misma prioridad que la multiplicación y la división.

Orden de las operaciones

1. Paréntesis

2. Multiplicación, división, resto

3. Suma, resta.

3.6 Conversión de tipo numérico

No es posible mezclar y combinar tipos de números al ejecutar operaciones matemáticas.

Se tienen dos números: uno se declara como `Int` y el otro como `Double`. Si se desean operar dichos números realizando una multiplicación es necesario crear un nuevo valor haciendo uso de un prefijo.

Otros operadores

Este tipo de operadores comúnmente son utilizados para realizar verificaciones y son los siguientes:

`==` igual que

`<` menor que

`>` mayor que

`!=` es distinto a

`<=` mayor o igual que

`>=` menor o igual que

Operadores lógicos

Este tipo de operadores devuelven valores verdaderos y falsos segun sea el caso.

Operador AND: &&
Operador OR: ||

04. CADENAS Y CARACTERES



- **4.1 Concatenación e interpolación**
- **4.2 Equivalencia y comparación de cadenas**
- **4.3 Unicode**

Cadenas o caracteres

Una cadena es una colección de caracteres. Las cadenas en Swift están representadas por el tipo String. Los caracteres individuales son del tipo Character.

La manera de definir una cadena sea constante o variable es utilizando un literal de cadena que no es más que el uso de comillas dobles.

4.1 Concatenación e interpolación

Concatenar: Unión de dos o más elementos de manera continua.

Interpolar: Intercalar o insertar elementos ajenos en un texto escrito.

+ concatenación
/() interpolación

4.2 Equivalencia y comparación de cadenas

Al igual que con los números, se puede verificar la igualdad entre dos cadenas mediante el uso del operador ==.

Métodos para verificar la equivalencia entre cadenas:

- 01. lowercased()
- 02. uppercased()
- 03. hasPrefix()
- 04. hasSuffix()
- 05. contains()

4.3 Unicode

Estándar que permite trabajar con más de 120,000 caracteres distintos utilizados en diferentes lenguas, así como emojis, símbolos y caracteres especiales.

Sea cual sea el símbolo, letra o emoji que se utilice Unicode garantiza que de manera individual su extensión sea igual a uno.



05. COLECCIONES

- **5.1 Arreglos**
- **5.2 Tipos de arreglos**
- **5.3 Trabajar con arreglos**
- **5.4 Diccionesarios**
- **5.5 Agregar/eliminar/modificar un diccionario**
- **5.6 Acceder a un diccionario**

COLECCIONES

Introducción

Hasta ahora hemos trabajado con elementos individuales, pero en múltiples ocasiones tendremos que trabajar con un grupo de datos como un todo, una colección en Swift te permite hacer referencia a varios objetos a la vez por ejemplo podrías manejar las múltiples casas en una unidad habitacional con una sola colección.

Swift define dos tipos de colecciones, los **arreglos** y los **diccionarios**. Cada tipo proporciona un método único para interactuar con varios objetos, aunque pueden compartir ciertas funciones.

COLECCIONES

5.1 Arreglos

El tipo de colección más común en Swift es un arreglo, que almacena una lista ordenada de valores del mismo tipo. Cuando declaras un arreglo, puedes especificar qué tipo de valores se mantendrán en la colección, o puedes dejar que el sistema de inferencia de tipo descubra el tipo.

```
var names: [String] = ["Anne", "Gary", "Keith"]
```

```
var names = ["Anne", "Gary", "Keith"]
```

```
let numbers = [4, 5, 6]
if numbers.contains(5) {
    print("Hay un 5")
}
```

COLECCIONES

5.2 Tipos de arreglos

Un arreglo es como una canasta: puede comenzar estando vacía y puedes llenarla con valores en un momento posterior. Pero si un literal de arreglo no contiene ningún valor, ¿cómo se puede inferir su tipo? Puedes declarar el tipo de un arreglo utilizando una anotación de tipo, una anotación de tipo de colección o un inicializador de arreglo.

```
var myArray: [Int] = []
```

Anotación de tipo tradicional

```
var myArray: Array<Int> = []
```

Anotación de tipo Colección especial

```
var myArray = [Int]()
```

**Anotación de tipo tradicional,
con inicializador de objetos Int**

COLECCIONES

5.3 Trabajar con arreglos

Por ejemplo, digamos que quieres un arreglo que se rellene con 100 ceros. En la forma de literal de arreglo, tendrías que ingresar los 100 ceros y sería fácil equivocarse al contarlos.

```
var myArray = [Int](repeating: 0, count: 100)
```

“Para averiguar el número de elementos dentro de un arreglo, puedes utilizar su propiedad count. ¿Qué sucede si quisieras verificar si el arreglo está vacío? En lugar de comparar count con 0, puedes verificar la propiedad isEmpty del arreglo, que devuelve Bool:

```
let count = myArray.count  
if myArray.isEmpty { }
```

COLECCIONES

5.3 Trabajar con arreglos

Para recuperar un valor particular de una colección, tienes que especificar dentro de corchetes el índice del valor al que deseas acceder. Los valores de un arreglo siempre están indexados a 0, lo que significa que el primer elemento en un arreglo tiene un índice de 0.

```
let firstName = names[0]
```

También puede usar la sintaxis de subíndice para cambiar un valor en un índice determinado. Al colocar el subíndice en el lado izquierdo del signo =, puedes configurar un valor particular en lugar de acceder al valor actual

```
names[1] = "Paul"
```


COLECCIONES

5.3 Trabajar con arreglos

Después de definir un arreglo y de configurar algunos valores, con frecuencia, querrás agregar valores nuevos. Puedes utilizar la función `append` del arreglo de la variable para agregar un elemento nuevo al final del arreglo. Para agregar varios elementos a la vez, utiliza el operador `+=`

```
var names = ["Amy"]  
names.append("Joe")  
names += ["Keith", "Jane"]  
print(names) // ["Amy", "Joe", "Keith", "Jane"]
```

```
names.insert("Bob", at: 0)
```

COLECCIONES

5.3 Trabajar con arreglos

“De manera similar, la función `remove(at:)` trabaja para eliminar un elemento en un índice especificado. A diferencia de `insert(_: at:)`, no necesitas ingresar el valor, la función devuelve el elemento eliminado de manera predeterminada, como en el ejemplo que se muestra a continuación. Otro método es `removeLast()`, que elimina el último elemento del arreglo, lo que descarta la necesidad de calcular el índice. Por último, el método `removeAll()` eliminará todos los elementos del arreglo.

```
var names = ["Amy", "Brad", "Chelsea", "Dan"]  
let chelsea = names.remove(at: 2)  
let dan = names.removeLast()  
names.removeAll()
```

COLECCIONES

5.3 Trabajar con arreglos

wift también te permite crear un nuevo arreglo al juntar dos arreglos existentes del mismo tipo.”

```
var myNewArray = firstArray + secondArray
```

Utiliza la sintaxis de literal de arreglo para colocar dos arreglos dentro de otro arreglo que los contenga. Puedes utilizar la sintaxis de subíndice en el arreglo contenedor para acceder a uno de los arreglos anidados. Para acceder a un elemento particular dentro de uno de los arreglos anidados, puedes utilizar dos conjuntos de subíndices.

```
let array1 = [1,2,3]
let array2 = [4,5,6]
let containerArray = [array1, array2] // [ [1,2,3], [4,5,6] ]
let firstArray = containerArray[0] // [1,2,3]
let firstElement = containerArray[0][0] // 1
```


COLECCIONES

5.4 Diccionarios

El segundo tipo de colección en Swift es un diccionario. Como un diccionario del mundo real que contiene una lista de palabras y sus definiciones, un diccionario de Swift es una lista de claves, cada una con un valor asociado. Cada referencia debe ser única, de la misma forma que cada palabra en el diccionario es única.

Puedes configurar un diccionario con un literal de diccionario: una lista de pares de referencias/valores separados por comas y rodeados por corchetes. Los dos puntos separan cada referencia y su valor resultante:

```
[key1: value1, key2: value2, key3: value3]
```

COLECCIONES

5.4 Diccionarios

“De la misma forma que en el arreglo, el tipo de diccionario se puede inferir según los tipos que se utilizan en el literal del diccionario.

“Digamos que quieres almacenar una lista de puntajes altos en un juego. Utilizarás el nombre del jugador como la referencia y el puntaje del jugador como el valor correspondiente. El tipo de diccionario se inferirá como un diccionario donde las referencias son de tipo String y los valores son de tipo Int. Esto puede representarse de las siguientes maneras [String: Int] o Dictionary<String, Int>

```
var scores = ["Richard": 500, "Luke": 400, "Cheryl": 800]
```

COLECCIONES

5.4 Diccionarios

Al igual que los demás tipos de colección, un diccionario tiene una propiedad `count` para determinar los pares de referencias/valores y una propiedad `isEmpty` para determinar si el diccionario no tiene pares de referencias/valores. La sintaxis para crear un diccionario vacío probablemente ya te resulte conocida:

```
var myDictionary = [String: Int]()  
  
var myDictionary = Dictionary<String, Int>()  
  
var myDictionary: [String: Int] = [:]
```

COLECCIONES

5.5 Agregar/eliminar/modificar un diccionario

La sintaxis de subíndice es especialmente útil con un diccionario. Como el orden en un diccionario Swift no importa, no hay índice y no hay riesgo de generar errores de subíndice asociados con los índices.

El siguiente ejemplo agrega un nuevo puntaje al diccionario de puntajes altos. Si la referencia “Oli” ya existe en el diccionario, este código reemplazará el valor antiguo con el nuevo:

```
scores["Oli"] = 399
```

COLECCIONES

5.5 Agregar/eliminar/modificar un diccionario

¿Qué sucede si quieres conocer si hay un valor antiguo en el diccionario antes de reemplazarlo? Puedes utilizar **updateValue(_:, forKey:)** para actualizar el diccionario y el valor que devuelva el método será igual al valor antiguo, si es que existía.

En el siguiente ejemplo, **oldValue** será igual al valor antiguo de **Richard** antes de la actualización. Si no existía un valor **oldValue** será **nil**. Más adelante aprenderás más sobre la palabra clave **nil**. Es una forma especial de representar la ausencia de un valor.

```
let oldValue = scores.updateValue(100, forKey: "Richard")
```


COLECCIONES

5.5 Agregar/eliminar/modificar un diccionario

SWIFT UTILIZA LA SINTAXIS “IF-LET” PARA DEJARTE EJECUTAR CÓDIGO SOLAMENTE SI SE DEVUELVE UN VALOR DEL MÉTODO. SI NO HABÍA UN VALOR EXISTENTE, EL CÓDIGO ENTRE LLAVES NO SE EJECUTARÁ:

```
if let oldValue = scores.updateValue(100, forKey: "Richard") {  
    print("El valor antiguo de Richard era \(oldValue)")  
}
```

COLECCIONES

5.5 Agregar/eliminar/modificar un diccionario

Para eliminar un elemento del diccionario, puedes utilizar sintaxis de subíndice al configurar el valor en nil. De una manera similar a la actualización de un valor, los diccionarios tienen un método `removeValue(forKey:)` si necesitas que devuelva el valor antiguo antes de eliminarlo:

```
var scores = ["Richard": 100, "Luke": 400, "Cheryl": 800]
scores["Richard"] = nil // ["Luke": 400, "Cheryl": 800]

if let removedValue = scores.removeValue(forKey: "Luke") {
    print("El puntaje de Luke era \(removedValue) antes de que dejara de jugar")
}
```

COLECCIONES

5.6 Acceder a un diccionario

Los diccionarios Swift te proporcionan dos propiedades que no se incluyen en otros tipos de colecciones. Puedes utilizar `keys` para devolver una lista de referencias dentro de un diccionario y `values` para devolver una lista de los valores. Si quieres utilizar estas colecciones posteriormente, tendrás que convertirlas en arreglos:

```
var scores = ["Richard": 500, "Luke": 400, "Cheryl": 800]

let players = Array(scores.keys) // ["Richard", "Luke", "Cheryl"]
let points = Array(scores.values) // [500, 400, 800]
```


COLECCIONES

5.6 Acceder a un diccionario

Para buscar un valor en particular en el diccionario, utiliza la sintaxis “if-let”. Si la referencia que especificas está en el diccionario, el resultado será el valor correspondiente a la referencia. Sin embargo, si la referencia no está en el diccionario, el código dentro de los corchetes no se ejecutará.

```
if let lukesScore = scores["Luke"] {  
    print(lukesScore)  
}  
  
if let henrysScore = scores["Henry"] {  
    print(henrysScore) // no se ejecutó; "Henry" no es una  
                        referencia en el diccionario  
}
```

Datos en la consola:

400

6. ESTRUCTURAS DE CONTROL



SWIFT

- **6.1 Operadores lógicos y de comparación**
- **6.2 Instrucciones "IF"**
- **6.3 Instrucciones "IF-ELSE"**
- **6.4 Valores Booleanos**
- **6.5 Instrucción "Switch"**
- **6.6 Operador Condicional Ternario**
- **6.7 Ciclos "For"**
- **6.8 Ciclos "While"**
- **6.9 Ciclos "Repeat-While"**
- **6.10 Instrucciones de transferencias de control**

ESTRUCTURAS DE CONTROL

Introducción

Escribirás un código que tome decisiones sobre qué líneas de código deben ejecutarse según los resultados del código ejecutado anteriormente. Esto se denomina flujo de control.

En esta lección, aprenderás a usar operadores lógicos para verificar las condiciones y usar instrucciones de flujo de control (if, if-else y switch) para elegir el código que se ejecutará como consecuencia.

ESTRUCTURAS DE CONTROL

6.1 Operadores lógicos y de comparación

Cada instrucción if usa un operador lógico o de comparación para decidir si algo es verdadero o falso. El resultado determina si se debe ejecutar el bloque de código u omitirlo.

Operador	Tipo	Descripción
==	Comparación	Dos elementos deben ser iguales.
!=	Comparación	Los valores no deben ser iguales entre sí.
>	Comparación	El valor de la izquierda debe ser mayor que el valor de la derecha.
>=	Comparación	El valor de la izquierda debe ser mayor o igual que el valor de la derecha.
<	Comparación	El valor de la izquierda debe ser menor que el valor de la derecha.
<=	Comparación	El valor de la izquierda debe ser menor o igual que el valor de la derecha.
&&	Lógico	Y: las instrucciones condicionales a la izquierda y a la derecha deben ser verdaderas (<code>true</code>).
	Lógico	O: la instrucción condicional a la izquierda o la instrucción condicional a la derecha debe ser verdadera (<code>true</code>).
!	Lógico	NO (NOT): devuelve el opuesto lógico de la instrucción condicional que sigue inmediatamente al operador.

ESTRUCTURAS DE CONTROL

6.2 Instrucciones "IF"

Si esta condición es verdadera, se debe ejecutar este bloque de código Si la condición no es verdadera, el programa omite el bloque de código.

```
let temperature = 100
if temperature >= 100 {
  print("El agua está hirviendo.")
}
```

Datos en la consola:

```
El agua está hirviendo.
```


ESTRUCTURAS DE CONTROL

6.3 Instrucciones "IF-ELSE"

¿qué sucede si la condición no es verdadera? Al agregar una cláusula “else” a una instrucción if, puedes especificar que se ejecute un bloque de código si la condición no es verdadera:

```
let temperature = 100
if temperature >= 100 {
  print("El agua está hirviendo.")
} else {
  print("El agua no está hirviendo.")
}
```

ESTRUCTURAS DE CONTROL

6.3 Instrucciones "IF-ELSE"

Puedes llevar esta idea aún más lejos. Mediante el uso de else if, puedes establecer que se ejecuten más bloques de código según cualquier cantidad de condiciones. El siguiente código verifica la posición de un atleta en una carrera y responde en consecuencia:

```
var finishPosition = 2

if finishPosition == 1 {
    print(";Felicitaciones, ganaste la medalla de oro!")
} else if finishPosition == 2 {
    print(";Quedaste en segundo lugar, ganaste una medalla de plata!")
} else {
    print("No ganaste ninguna medalla de oro ni de plata.")
}
```

ESTRUCTURAS DE CONTROL

6.4 Valores Booleanos

“Puedes asignar los resultados de una comparación lógica a una constante o una variable Bool para verificar o acceder al valor más adelante”

```
let number = 1000
let isSmallNumber = number < 10
// El número no es menor que 10, por lo que se le asigna
  a isSmallNumber un valor 'falso'
```

```
let speedLimit = 65
let currentSpeed = 72
let isSpeeding = currentSpeed > speedLimit
// CurrentSpeed es mayor que speedLimit, por lo que
  se le asigna a isSpeeding un valor 'verdadero'
```


ESTRUCTURAS DE CONTROL

6.4 Valores Booleanos

“También es posible invertir un valor Bool mediante el operador lógico NO (NOT), que se representa con !

```
var isSnowing = false

if !isSnowing {
    print("No está nevando.")
}
```

Datos en la consola:

No está nevando.

ESTRUCTURAS DE CONTROL

6.4 Valores Booleanos

“De la misma forma, puedes usar un operador lógico "&&" o "||"

```
let temperature = 70

if temperature >= 65 && temperature <= 75 {
  print("La temperatura es la adecuada.")
} else if temperature < 65 {
  print("Hace demasiado frío.")
} else {
  print("Hace demasiado calor.")
}
```

Datos en la consola:

La temperatura es la adecuada.

```
var isPluggedIn = false
var hasBatteryPower = true

if isPluggedIn || hasBatteryPower {
  print("Puedes usar tu computadora portátil.")
} else {
  print("🔌")
}
```

Datos en la consola:

Puedes usar tu computadora portátil.

ESTRUCTURAS DE CONTROL

6.5 Instrucción "Switch"

Swift tiene otra herramienta para controlar el flujo, conocida como instrucción switch, que es óptima para funcionar con muchas condiciones potenciales.

Una instrucción switch básica toma un valor con varias opciones y te permite ejecutar un código diferente en función de cada opción, o caso (case). También puedes proporcionar un caso predeterminado (default) para especificar un bloque de código que se ejecutará en todos los casos que no hayas definido específicamente.

ESTRUCTURAS DE CONTROL

6.5 Instrucción "Switch"

```
let numberOfWheels = 2
switch numberOfWheels {
case 0:
    print("¿Falta algo?")
case 1:
    print("Monociclo")
case 2:
    print("Bicicleta")
case 3:
    print("Triciclo")
case 4:
    print("Cuatriciclo")
default:
    print("¡Son muchas ruedas!")
}
```

ESTRUCTURAS DE CONTROL

6.5 Instrucción "Switch"

CUALQUIER INSTRUCCIÓN CASE DETERMINADA TAMBIÉN PUEDE EVALUAR VARIAS CONDICIONES A LA VEZ. POR EJEMPLO, EL SIGUIENTE CÓDIGO VERIFICA SI UN CARÁCTER ES UNA VOCAL O NO:

```
let character = "z"

switch character {
case "a", "e", "i", "o", "u":
    print("Este carácter es una vocal.")
default:
    print("Este carácter es una consonante.")
}
```


ESTRUCTURAS DE CONTROL

6.5 Instrucción "Switch"

CUANDO TRABAJES CON NÚMEROS, PUEDES USAR LA COINCIDENCIA DE INTERVALOS PARA VERIFICAR LA INCLUSIÓN EN UN RANGO. ECHA UN VISTAZO A LA SINTAXIS EN EL SIGUIENTE FRAGMENTO DE CÓDIGO:

```
switch distance {  
case 0...9:  
    print("Tu destino está cerca.")  
case 10...99:  
    print("Tu destino está a una distancia intermedia de aquí.")  
case 100...999:  
    print("Tu destino está lejos de aquí.")  
default:  
    print("¿Estás seguro de que quieres viajar tan lejos?")  
}
```

ESTRUCTURAS DE CONTROL

6.6 Operador Condicional Ternario

Un caso de uso interesante (y muy común) para una instrucción if es establecer una variable o devolver un valor. Si una determinada condición es verdadera, se le asigna un valor determinado a una variable. Si la condición es falsa, se le asigna otro valor a la variable.

```
var largest: Int

let a = 15
let b = 4

if a > b {
    largest = a
} else {
    largest = b
}
```

ESTRUCTURAS DE CONTROL

6.6 Operador Condicional Ternario

El operador ternario tiene tres partes:

1. Una pregunta con respuesta de verdadero o falso.
2. Un valor si la respuesta a la pregunta es verdadera.
3. Un valor si la respuesta a la pregunta es falsa.

```
var largest: Int
```

```
let a = 15
```

```
let b = 4
```

```
largest = a > b ? a : b
```


ESTRUCTURAS DE CONTROL

Ciclos introducción

Es probable que se te ocurran muchas tareas cotidianas que sigues haciendo hasta que se cumpla una condición. Podrías seguir llenando una botella de agua hasta que esté llena o seguir haciendo la tarea hasta completarla.

Los escenarios que requieren la finalización y repetición de una tarea pueden realizarse en programación por medio de ciclos. En esta lección, aprenderás a crear ciclos en Swift, controlar las condiciones de la ejecución de los ciclos y especificar cuándo deben detenerse.

ESTRUCTURAS DE CONTROL

6.7 Ciclos "For"

Un ciclo "for-in" ejecuta un conjunto de instrucciones para cada elemento dentro de un rango, una secuencia o una colección.

```
for index in 1...5 {  
    print("Este es el número \ (index)")  
}
```

Datos en la consola:

```
Este es el número 1  
Este es el número 2  
Este es el número 3  
Este es el número 4  
Este es el número 5
```

ESTRUCTURAS DE CONTROL

6.7 Ciclos "For"

El operador "..." se conoce como el operador de rango cerrado, que es como se define un rango de valores que va desde **x** hasta **y** e incluye tanto **x** como **y** en el rango. Existe un operador complementario (**..<**) conocido como el operador de rango semiabierto. Estos rangos van desde **x** hasta **y**, pero no incluyen **y**.

En el código anterior, **index** es una constante que está disponible para personalizar el trabajo realizado dentro de las llaves (el ciclo "for-in").

ESTRUCTURAS DE CONTROL

6.7 Ciclos "For"

Pero supongamos que tu resultado no necesita usar los valores en el rango. Si solo necesitas realizar una serie de pasos un determinado número de veces, puedes omitir la asignación de un valor a una constante y reemplazar su nombre por un " "

```
for _ in 1...3 {  
    print("¡Hola!")  
}
```

Datos en la consola:

¡Hola!

¡Hola!

¡Hola!

ESTRUCTURAS DE CONTROL

6.7 Ciclos "For"

Puedes usar la misma sintaxis "for-in" para iterar sobre cada elemento de un arreglo.

Debido a que String es un tipo de colección, tiene una funcionalidad similar a la de un arreglo. Puedes usar un ciclo "for-in" para iterar cada carácter en una String:

```
let names = ["Joseph", "Cathy", "Winston"]
for name in names {
    print("Hola, \(name)")
}
```

Datos en la consola:

```
Hola, Joseph
Hola, Cathy
Hola, Winston
```

```
for letter in "ABCD" {
    print("La letra es \(letter)")
}
```

Datos en la consola:

```
La letra es la A
La letra es la B
La letra es la C
La letra es la D
```


ESTRUCTURAS DE CONTROL

6.7 Ciclos "For"

¿Qué sucede si necesitas el índice de cada elemento además de su valor? Puedes usar el método `enumerated()` de un arreglo o una cadena para devolver una tupla, un tipo especial que puede reunir una lista ordenada de valores entre paréntesis, que contiene el índice y el valor de cada elemento:

```
for (index, letter) in "ABCD".enumerated() {  
    print("\(index): \(letter)")  
}
```

Datos en la consola:

0: A

1: B

2: C

3: D

ESTRUCTURAS DE CONTROL

6.7 Ciclos "For"

Como alternativa, puedes usar el operador de rango semiabierto para hacer lo mismo:

```
let animals = ["León", "Tigre", "Oso"]  
for index in 0..  
animals.count {  
    print("\(index): \n(animals[index])")  
}
```

Datos en la consola:

0: León

1: Tigre

2: Oso

ESTRUCTURAS DE CONTROL

6.7 Ciclos "For"

Si usas un ciclo "for-in" con un diccionario, el ciclo genera una tupla que contiene la clave y el valor de cada entrada. Debido a que por lo general se accede a un diccionario especificando una clave, el ciclo no garantiza ningún orden en particular de los elementos a medida que trabaja a través del diccionario:

```
let vehicles = ["monociclo": 1, "bicicleta": 2, "triciclo": 3,
"cuatriciclo": 4]
for (vehicleName, wheelCount) in vehicles {
    print("Un(Una) \(vehicleName) tiene \(wheelCount) rueda(s)")
}
```

Datos en la consola:

```
Un monociclo tiene 1 rueda
Una bicicleta tiene 2 ruedas
Un triciclo tiene 3 ruedas
Un cuatriciclo tiene 4 ruedas
```


ESTRUCTURAS DE CONTROL

6.8 Ciclos "While"

Un ciclo “while” continuará ejecutando el ciclo hasta que su condición especificada deje de ser verdadera.

```
var numberOfLives = 3  
  
while numberOfLives > 0 {  
    playMove()  
    updateLivesCount()  
}
```

ESTRUCTURAS DE CONTROL

6.8 Ciclos "While"

Swift comprueba la condición antes de que se ejecute cada ciclo, lo que significa que es posible omitir el ciclo por completo si nunca se cumple la condición. En el ejemplo anterior, nada actualiza el valor de `numberOfLives`, por lo que la condición siempre resulta ser `true` y continúa para siempre.

```
var numberOfLives = 3

while numberOfLives > 0 {
    print("Todavía tengo \(numberOfLives) vidas.")
}
```

```
var numberOfLives = 3
var stillAlive = true
while stillAlive {
    numberOfLives -= 1
    if numberOfLives == 0 {
        stillAlive = false
    }
}
```

ESTRUCTURAS DE CONTROL

6.9 Ciclos "Repeat-While"

Un ciclo "repeat-while" es similar al ciclo "while", pero esta sintaxis ejecuta el bloque una vez antes de comprobar la condición.

var

```
var steps = 0
let wall = 2 // hay una pared después de dos pasos

repeat {
  print("Paso")

  steps += 1

  if steps == wall {
    print(";Tropezaste con una pared!")
    break
  }
} while steps < 10 // máximos en esta dirección
```

Datos en la consola:

Paso

Paso

;Tropezaste con una pared!

ESTRUCTURAS DE CONTROL

6.10 Instrucciones de transferencias de control

Es posible que haya situaciones en las que quieras detener la ejecución de un ciclo desde el cuerpo de este. La palabra clave `break` de Swift desglosará la ejecución del código dentro del ciclo y comenzará a ejecutar cualquier código definido después del ciclo.

```
// Imprime -3 hasta 0
for counter in -3...3 {
    print(counter)
    if counter == 0 {
        break
    }
}
```

Datos en la consola:

```
-3
-2
-1
0
```

ESTRUCTURAS DE CONTROL

6.10 Instrucciones de transferencias de control

Además, es posible que haya situaciones en las que quieras pasar a la iteración siguiente en un ciclo. Mientras que la palabra clave `break` finalizará el ciclo por completo, `continue` avanzará a la siguiente iteración. Por ejemplo, puedes tener un arreglo en el que cada elemento es del tipo `Person` y quieres ejecutar un ciclo a través del arreglo y enviar un correo electrónico a todas las personas mayores de 18 años:

```
for person in people {  
  if person.age < 18 {  
    continue  
  }  
  
  sendEmail(to: person)  
}
```


7. FUNCIONES



- **7.1 Definir una función**
- **7.2 Parámetros**
- **7.3 Valores de parámetros predeterminados**
- **7.4 Valores de devolución**

Abstracción

Acción que permite definir algo que es complejo de una manera más sencilla.

Las funciones, principalmente aquellas que el que el propio programador diseña permiten reutilizar código.

7.1 Definir una función

```
func nombreFuncion (parámetros) -> valorDevolucion {  
    //cuerpo de la función  
    retorno  
}
```

```
nombreFuncion()
```


7.2 Parámetros

Los parámetros son los elementos con los que una función trabaja. En Swift, una función puede o no tener parámetros. Ya sea uno o varios los parámetros estos se ubican dentro de un paréntesis en el que escribe el nombre del parámetro seguido de dos puntos y el tipo de dato.

(suma: Int)

(suma: Int, resta: Int)

7.3 Valores de parámetros predeterminados

En ocasiones existen valores que no cambiarán y se utilizarán en repetidas ocasiones, asignar un valor predeterminado a un parámetro resultará bastante útil. (**puntaje: Int = 0**)

7.4 Valores de devolución

Una función puede devolver el valor de una nueva constante ubicada en el cuerpo de la función o bien no crear ninguna constante y realizar la operación en la misma línea que el return.

8. CLOSURES



SWIFT

- **8.1 Sintaxis**
- **8.2 Pasar closures como argumentos**
- **8.3 Devolución de valores en closures**
- **8.4 Closures como parámetros**

CLOSURES

8.1 Sintaxis

Swift nos permite usar funciones como cualquier otro tipo, como cadenas y números enteros. Esto significa que puedes crear una función y asignarla a una variable, llamar a esa función usando esa variable e incluso pasar esa función a otras funciones como parámetros.

Las funciones utilizadas de esta manera se llaman cierres, y aunque funcionan como funciones, se escriben de manera un poco diferente.

Comencemos con un ejemplo simple que imprime un mensaje:

```
let driving = {  
  print("Estoy manejando un carro")  
}
```

Eso crea una función sin nombre y asigna esa función a "driving".
Ahora puedes llamar a driving() como si fuera una función normal, así:

```
driving()
```

CLOSURES

8.2 Aceptar parametros en closures

Para hacer que un cierre acepte parámetros, hay que ponerlos dentro de los paréntesis justo después del corchete de apertura, luego escribir " in " para que Swift sepa que el cuerpo principal del cierre está comenzando.

Por ejemplo, podríamos hacer un cierre que acepte una cadena de nombre de lugar como su único parámetro, como este:

```
let driving = {(place: String) in
    print("Estoy viajando a \(place) en carro")
}
```

Una de las diferencias entre las funciones y los closures es que no se utilizan etiquetas de parámetros al ejecutar cierres. Entonces, para llamar a `driving()` ahora escribiríamos esto:

```
driving("London")
```

Consola:

```
Estoy viajando a London en carro
```


CLOSURES

8.3 Devolución de valores en closures

Los cierres también pueden devolver valores, y se escriben de manera similar a los parámetros: los escribes dentro de tu cierre, directamente antes de la palabra clave **in**.

Para demostrar esto, vamos a tomar nuestro closure `driving()` y hacer que devuelva su valor en lugar de imprimirlo directamente.

Queremos un cierre que devuelva una cadena en lugar de imprimir el mensaje directamente, por lo que tenemos que usar " `-> String` " antes del **in**, y luego usar `return` como una función normal:

```
let drivingReturn = {(place: String) -> String in
    return "Estoy viajando a \(place) en carro"
}
```

Ahora podemos ejecutar ese closure e imprimir su valor de retorno:

```
let message = drivingReturn("London")
```

```
print(message)
```

Consola:

```
Estoy viajando a London en carro
```

CLOSURES

8.4 Closures como parametros

Debido a que los cierres se pueden usar como cadenas o enteros, puedes pasarlos a funciones.

Primero, aquí está nuestro cierre básico de `driving()` de nuevo

```
let driving = {  
    print("Estoy manejando un carro")  
}
```

Si quisiéramos pasar ese closure a una función para que se pueda ejecutar dentro de esa función, especificaríamos el tipo de parámetro como `() -> Void`. Eso significa " acepta sin parámetros y devuelve Void", la forma en que Swift dice "nada".

CLOSURES

8.3 Closures como parametros

Por lo tanto, podemos escribir una función `travel()` que acepta diferentes tipos de acciones de viaje e imprime un mensaje antes y después:

```
func travel(action: () -> void) {  
    print("Estoy alistándome para salir")  
    action()  
    print("He llegado!")  
}
```

Ahora podemos llamar a eso usando nuestro closure `driving` así:

```
travel (action: driving)
```

9. ENUMERATIONS



SWIFT

- **9.1 Flujo de control**
- **9.2 Ventajas de la seguridad de tipo**

ENUMERATIONS

Una enumeración, o enum, es un tipo especial de Swift que permite representar un conjunto de opciones con nombre. Veremos cuándo se usan comúnmente las enumeraciones, cómo definir una enumeración y cómo trabajar con enumeraciones mediante instrucciones switch.

Pensemos en las direcciones de la app de una brújula: norte, sur, este y oeste. La app puede ayudar a orientar al usuario hacia cualquiera de esas cuatro direcciones. La aguja de una brújula siempre apunta al norte, pero el rumbo de la brújula se mueve cuando el usuario se mueve. El rumbo ayuda al usuario a determinar en qué dirección está orientado.

ENUMERATIONS

Se define una nueva enumeración con la palabra clave enum. El código siguiente define una enum para seguir la dirección de una brújula:

```
enum CompassPoint = {  
    case norte  
    case sur  
    case este  
    case oeste  
}
```

La enum define el tipo, y las opciones case definen los valores disponibles que se permiten con el tipo. Es una práctica recomendada poner en mayúsculas el nombre de la enumeración y poner en minúsculas las opciones case.

ENUMERATIONS

También puedes definir los casos disponibles, separados por comas, en una sola línea:

```
enum CompassPoint = {  
    case norte, sur, este, oeste  
}
```

Una vez que se definió la enumeración, puedes comenzar a usarla como cualquier otro tipo en Swift. Solo hay que especificar el tipo de enumeración junto con el valor:

```
var compassDir = CompassPoint.norte  
  
var compassDir: CompassPoint = .norte
```


ENUMERATIONS

Ahora que el tipo de `compassDir` está establecido, puedes cambiar el valor a otro punto de la brújula con la notación de puntos más corta:

```
compassDir = .norte
```

ENUMERATIONS

9.1 Flujo de control

En la lección sobre el flujo de control, aprendieron a usar las instrucciones `if` y `switch` para responder a los valores `Bool`. Podemos usar la misma lógica de flujo de control cuando se trabaja con diferentes casos de una enumeración.

El siguiente código de abajo imprime una oración diferente en función de qué `CompassPoint` se establece en la constante `compassDir`:

ENUMERATIONS

9.1 Flujo de control

Switch

```
let compassDir: CompassPoint =  
    .west  
  
switch compassDir {  
    case .north:  
        print("Me dirijo al norte.")  
    case .east:  
        print("Me dirijo al este.")  
    case .south:  
        print("Me dirijo al sur.")  
    case .west:  
        print("Me dirijo al oeste.")  
}
```

ENUMERATIONS

9.1 Flujo de control

If

```
let compassDir: CompassPoint = .west

if compassDir == .west {
    print("Me dirijo al oeste.")
}
```

ENUMERATIONS

9.2 Ventajas de seguridad de tipo

Las enumeraciones son especialmente importantes en Swift porque te permiten representar información, como cadenas o números con seguridad de tipo.

Imagina un conjunto de datos que representa películas de géneros específicos. Antes de aprender sobre las enumeraciones, es posible que hayas definido una estructura simple de películas como la siguiente:

```
struct Movie{  
    var name: String  
    var releaseYear: Int  
    var genre: String  
}
```


ENUMERATIONS

9.2 Ventajas de seguridad de tipo

Teniendo en cuenta esta definición, se usaría una String al establecer el género:

```
let movie = Movie(name: "Pinocho", releaseYear: 2022,  
genre: "Stop notion")
```

Pueden notar el error?

Genre es propenso a todos los errores a los que se enfrentan los valores String, uno de los cuales es la ortografía incorrecta. Imagina que escribieras un código para encontrar todas las películas del género “Stop Motion”. Faltaría Pinocho.

Como práctica recomendada, podrías asignarle a genre un valor de una enumeración llamada Genre.

ENUMERATIONS

9.2 Ventajas de seguridad de tipo

```
enum Genre {  
    case stopMotion, acción, romance, documental, biografía,  
    thriller  
}
```

```
struct Movie {  
    var name: String  
    var releaseYear: Int  
    var genre: Genre  
}
```

```
let movie = Movie(name: "Pinocho", releaseYear: 2022,  
genre: .stopMotion)
```

ENUMERATIONS

9.2 Ventajas de seguridad de tipo

Este código es mucho menos propenso a errores. El compilador refuerza la seguridad al exigirte que elijas un case de la enumeración Genre cuando inicializas una nueva película.

Las enumeraciones son una herramienta sumamente poderosa en Swift. Las usarás cada vez que quieras agregar seguridad de tipo donde, de otro modo, podrías usar cadenas o números.

10. ESTRUCTURAS



SWIFT

- **10.1 Instancias**
- **10.2 Inicializadores**
- **10.3 Métodos de instancia**
- **10.4 Métodos mutantes**
- **10.5 propiedades calculadas**
- **10.6 Observadores de propiedad**
- **10.7 Propiedades y métodos de tipo**
- **10.8 Copia**
- **10.9 Self**

ESTRUCTURAS

En su forma más simple, una estructura es un grupo con nombre de una o más propiedades que componen un tipo. Las propiedades representan la información sobre una instancia de la estructura.

Se define una estructura usando la palabra clave `struct` junto con un nombre único. Luego, se pueden definir las propiedades como parte de la `struct` enumerando las instrucciones de constantes o variables con la anotación de tipo apropiada. Por lo general, se ponen en mayúsculas los nombres de los tipos y se usa camel case en los nombres de las propiedades.

ESTRUCTURAS

Considera la simple declaración de una estructura Person con una propiedad name:

```
struct Person {  
    var name: String  
}
```

La declaración anterior simplemente define las propiedades de un tipo Person. No tiene un valor en sí misma. Para ello, hay que crear una instancia del tipo Person.

ESTRUCTURAS

Luego, puedes acceder a los datos almacenados en sus propiedades, como el `name` de la persona, usando la sintaxis de puntos.

```
let firstPerson = Person(name: "Jazmin")  
  
print(firstPerson.name)
```

Salida:

Jazmin

ESTRUCTURAS

Puedes agregar funcionalidad a una estructura mediante un método. Un método es una función que se asigna a un tipo específico. En este ejemplo, nuestras instancias `Person` ahora puede tener `sayHello()`.

```
struct Person {  
    var name: String  
    func sayHello(){  
        print("¡Hola! Mi nombre es \(name)!")  
    }  
}
```


ESTRUCTURAS

Ahora puedes llamar al método de instancia directamente con la sintaxis de puntos.

```
let person = Person(name: "Jazmin")  
person.sayHello()
```

Salida:

```
¡Hola! Mi nombre es Jazmin!
```

ESTRUCTURAS

10.1 Instancias

Acabas de aprender que una estructura define un nuevo tipo. Para usar ese tipo, debes crear una instancia de este mediante un proceso llamado inicialización. Después de la inicialización, cada instancia hereda todas las propiedades y características de la estructura.

En el siguiente ejemplo, tanto myShirt como yourShirt son objetos Shirt con propiedades size y color. Pero cada una es una camiseta independiente con valores distintos para cada propiedad y, por tanto, una instancia independiente del tipo Shirt.

ESTRUCTURAS

10.1 Instancias

```
struct Shirt {  
    var size: Size  
    var color: Color  
}
```

```
let myShirt = Shirt(size: .xl, color: .blue)
```

```
// Crea una instancia de una camiseta.
```

```
let yourShirt = Shirt(size: .m, color: .red)
```

```
// Crea una instancia independiente de una camiseta.
```


ESTRUCTURAS

Aquí tenemos otro ejemplo de definición de una estructura, esta vez con funciones agregadas. La estructura define los atributos y las funciones de un objeto Car y describe firstCar y secondCar como dos instancias.

```
struct Car {  
    var make: String  
    var model: String  
    var year: Int  
    var topSpeed: Int  
  
    func startEngine() {  
        print("El motor del \$(make) \$(model) \$(year) ha arrancado.")  
    }  
    func drive() {  
        print("El \$(make) \$(model) \$(year) se está moviendo.")  
    }  
    func park() {  
        print("El \$(make) \$(model) \$(year) está estacionado.")  
    }  
}
```

```
let firstCar = Car(make: "Honda", model: "Civic", year: 2010,  
topSpeed: 120)
```

```
let secondCar = Car(make: "Tesla", model: "Model Y", year: 2022,  
topSpeed: 250)
```

```
firstCar.startEngine()  
secondCar.drive()
```

Salida:

El motor del Honda Civic 2010 ha arrancado.
El Tesla Model Y 2022 se está moviendo.

ESTRUCTURAS

10.2 Inicializadores

Todos los tipos de Swift cuentan con un inicializador, que es similar a una función que devuelve una nueva instancia del tipo. El inicializador predeterminado es `init()`.

Las instancias creadas a partir del inicializador predeterminado tienen un valor predeterminado.

La String predeterminada es: ""

El Int predeterminado es: 0

El Double predeterminado es: 0.0

El Bool predeterminado es: false

```
var string = String.init()  
var int = Int.init()  
var double = Double.init()  
var bool = Bool.init()
```

ESTRUCTURAS

10.2 Inicializadores

Pero hay una versión abreviada del inicializador predeterminado que es mucho más común. El siguiente fragmento es más conciso, pero funciona igual que el ejemplo anterior:

```
var string = String()  
var int = Int()  
var double = Double()  
var bool = Bool()
```

Siempre que definas un nuevo tipo, debes tener en cuenta cómo vas a crear nuevas instancias. En esta lección se exponen diferentes enfoques para inicializar los valores de propiedad

ESTRUCTURAS

10.3 Métodos de instancia

Los métodos de instancia son funciones a las que se puede llamar en instancias específicas de un tipo. Proporcionan formas de acceder a las propiedades de la estructura y modificarlas, y agregan funciones relacionadas con el propósito de la instancia.

Considera una estructura `Size` con un método de instancia `area()` que calcula el área de una instancia específica multiplicando su ancho y su alto:

ESTRUCTURAS

10.3 Métodos de instancia

```
struct Size {  
    var width: Double  
    var height: Double  
  
    func area() -> Double {  
        width * height  
    }  
}
```

```
let someSize = Size(width: 10.0, height: 5.5)  
let area = someSize.area()
```

La instancia `someSize` es del tipo `Size`, y `width` y `height` son sus propiedades. El `area()` es un método de instancia al que se puede llamar en todas las instancias del tipo `Size`.

ESTRUCTURAS

10.4 Métodos mutantes

En ocasiones querremos actualizar los valores de propiedad de una estructura dentro de un método de instancia. Para ello, se debe agregar la palabra clave **mutating** antes de la función.

En el siguiente ejemplo, una estructura simple almacena datos de millaje sobre un objeto Car específico. Antes de ver el código, considera qué datos necesita almacenar el contador de millas y qué acciones necesita realizar.

- Almacenar el conteo de millas para que se muestre en un odómetro.
- Incrementar el contador de millas para actualizar el millaje cuando el auto circule.
- Posiblemente restablecer el contador de millas si el auto supera el número de millas que se pueden mostrar en el odómetro.

ESTRUCTURAS

10.4 Métodos mutantes

```
struct Tacometro {  
    var count: Int = 0  
  
    mutating func increment() {  
        count += 1  
    }  
  
    mutating func increment(by amount: Int) {  
        count += amount  
    }  
  
    mutating func reset() {  
        count = 0  
    }  
}
```


ESTRUCTURAS

10.4 Métodos mutantes

```
var tacometro = Tacometro() // tacometro.count es 0 de forma  
predeterminada
```

```
tacometro.increment() // odometer.count se incrementa a 1
```

```
tacometro.increment(by: 15) // odometer.count se incrementa a 16
```

```
tacometro.reset() // odometer.count se restablece a 0
```

La instancia odometer es del tipo **Tacómetro**, **increment()** e **increment(by:)** son métodos de instancia que agregan millas a la instancia. El método de instancia **reset()** restablece la cuenta de millaje a cero.

ESTRUCTURAS

10.5 Propiedades calculadas

Consideremos el ejemplo de Temperature. Aunque la mayor parte del mundo usa la escala Celsius para medir la temperatura, en algunos lugares (como Estados Unidos) se usa la escala Fahrenheit, y en ciertas profesiones se usa la Kelvin. Por lo tanto, podría ser útil que la estructura Temperature admita los tres sistemas de medición.

```
struct Temperature {  
    var celsius: Double  
    var fahrenheit: Double  
    var kelvin: Double  
}
```

ESTRUCTURAS

10.5 Propiedades calculadas

Imagina que hubieras usado el inicializador de miembros para esta estructura:

```
let temperature = Temperature(celsius: 0, fahrenheit:  
32.0,  
kelvin: 273.15)
```

Cada vez que escribieras un código para inicializar un objeto Temperature, tendrías que calcular cada temperatura y pasar todos esos valores como parámetros.

Una de las alternativas consistiría en agregar varios inicializadores que manejen los cálculos.

El enfoque anterior, que usa varios inicializadores, implica la administración de una gran cantidad de estados o información. Cada vez que la temperatura cambia, tendrías que actualizar las tres propiedades. Este enfoque es propenso a errores y sería frustrante para cualquier programador.

ESTRUCTURAS

10.5 Propiedades calculadas

Swift proporciona un enfoque más seguro. Con las propiedades calculadas, puedes crear propiedades que pueden calcular su valor basándose en otras propiedades de instancia o en la lógica.

```
struct Temperature {  
    var celsius: Double  
  
    var fahrenheit: Double {  
        celsius * 1.8 + 32  
    }  
  
    var kelvin: Double {  
        celsius + 273.15  
    }  
}
```

ESTRUCTURAS

10.5 Propiedades calculadas

Para agregar una propiedad calculada, debes declarar la propiedad como una variable (porque su valor puede cambiar). También debes declarar el tipo de manera explícita. Luego, debes usar una llave abierta ({}) y una llave de cierre (}) para definir la lógica que calcula el valor que se devolverá.

Puedes acceder a las propiedades calculadas con la sintaxis de puntos, como lo harías con cualquier otra propiedad.

```
let currentTemperature = Temperature(celsius: 0.0)

print(currentTemperature.fahrenheit)

print(currentTemperature.kelvin)
```


ESTRUCTURAS

10.6 Observadores de propiedad

Swift permite observar cualquier propiedad y responder a los cambios en el valor. Estos observadores de propiedad son llamados cada vez que se establece el valor de una propiedad, incluso si este es el mismo que el valor actual de la propiedad. Hay dos cierres de observador que se pueden definir en cualquier propiedad dada: **willSet** y **didSet**.

En el siguiente ejemplo, se definió un **StepCounter** con una propiedad **totalSteps**. Se definieron los observadores de **willSet** y **didSet**. Siempre que se modifique **totalSteps**, se llamará primero a **willSet**, y tendrás acceso al nuevo valor que se establecerá como el valor de la propiedad en una constante llamada **newValue**. Una vez actualizado el valor de la propiedad, se llamará a **didSet** y podrás acceder al valor anterior de la propiedad mediante **oldValue**.

ESTRUCTURAS

10.6 Observadores de propiedad

```
struct StepCounter {  
    var totalSteps: Int = 0 {  
        didSet {  
            print("A punto de establecer totalSteps en \(newValue)")  
        }  
        didSet {  
            if totalSteps > oldValue {  
                print("Se agregaron \(totalSteps - oldValue) pasos")  
            }  
        }  
    }  
}
```

ESTRUCTURAS

10.6 Observadores de propiedad

Este es el resultado al modificar la propiedad `totalSteps` de un nuevo `StepCounter`:

```
var stepCounter = StepCounter()  
stepCounter.totalSteps = 40  
stepCounter.totalSteps = 100
```

Salida:

A punto de establecer `totalSteps` en 40

Se agregaron 40 pasos

A punto de establecer `totalSteps` en 100

Se agregaron 60 pasos

ESTRUCTURAS

10.7 Propiedades y métodos de tipo

Swift permite agregar propiedades y métodos de tipo, a los que se puede acceder o llamar en el propio tipo. Usa la palabra clave **static** para agregar una propiedad o un método a un tipo.

Las propiedades de tipo son útiles cuando una propiedad está relacionada con el tipo, pero no es una característica de una propia instancia.

ESTRUCTURAS

10.7 Propiedades y métodos de tipo

En el siguiente ejemplo, se define una estructura `Temperature` que tiene una propiedad estática llamada `boilingPoint`, que es un valor constante para todas las instancias de `Temperature`.

```
struct Temperature {  
    static var boilingPoint = 100  
}
```

Se accede a las propiedades del tipo con la sintaxis de puntos en el nombre del tipo

```
let boilingPoint = Temperature.boilingPoint
```

ESTRUCTURAS

10.7 Propiedades y métodos de tipo

Los métodos de tipo son similares a las propiedades de tipo. Usa un método de tipo cuando la acción esté relacionada con el tipo, pero no sea algo que deba realizar una instancia específica de este.

La estructura `Double` definida en la biblioteca estándar de Swift, contiene un método estático llamado `minimum` que devuelve el menor de sus dos parámetros.

```
let smallerNumber = Double.minimum(100.0, -1,000.0)
```

ESTRUCTURAS

10.8 Copia

Si asignamos una estructura a una variable o pasamos una instancia como parámetro a una función, los valores se copian. Por lo tanto, las variables independientes son instancias independientes del valor, lo que significa que si se cambia un valor no se cambia el otro.

```
var someSize = Size(width: 250, height: 1000)
var anotherSize = someSize
```

```
someSize.width = 500
```

```
print(someSize.width)
print(anotherSize.width)
```

500

250

ESTRUCTURAS

10.9 Self

En Swift, `self` hace referencia a la instancia actual del tipo. Se puede usar dentro de un método de instancia o una propiedad calculada para hacer referencia a su propia instancia.

```
struct Car {  
    var color: Color  
  
    var description: String {  
        return "Este es un auto \(self.color)."  
    }  
}
```


ESTRUCTURAS

10.9 Self

Muchos lenguajes requieren el uso de self para hacer referencia a propiedades o métodos de instancia dentro de la definición del tipo. El compilador de Swift reconoce cuando los nombres de propiedades o métodos existen en el objeto actual, y hace que el uso de self sea opcional.

Este ejemplo es funcionalmente el mismo que el ejemplo anterior.

```
struct Car {  
    var color: Color  
  
    var description: String {  
        return "Este es un auto \(color)."  
    }  
}
```

ESTRUCTURAS

10.9 Self

El uso de self es necesario dentro de inicializadores que tienen nombres de parámetros que coinciden con los nombres de las propiedades.

```
struct Temperature {  
    var celsius: Double  
  
    init(celsius: Double) {  
        self.celsius = celsius  
    }  
}
```

11. CLASES



SWIFT

- **11.1 Herencia**
- **11.2 Referencias**
- **11.3 Inicializadores de miembros**
- **11.4 Clase o estructura**

12. OPCIONALES



SWIFT

- **12.1 Nil (Nulo)**
- **12.2 Especificar el tipo de un opcional**
- **12.3 Trabajar con valores opcionales**
- **12.4 Funciones y opcionales**
- **12.5 Inicializadores falibles**
- **12.6 Encadenamiento opcional**
- **12.7 Opcionales extraídos de forma implícita**

OPCIONALES

12 Introducción

Una de las principales ventajas de **swift** es la capacidad de inferencia de datos. Cuando una función **puede o no regresar datos**, **Swift** te obliga a tratar adecuadamente ambos escenarios posibles.

Swift usa una sintaxis única, llamada **opcionales**, para manejar este tipo de casos. En esta lección, aprenderás a usar los **opcionales** para manejar adecuadamente las situaciones en las que los datos pueden o no existir.

OPCIONALES

12.1 Nil (Nulo)

Los opcionales son útiles en situaciones en las que **puede o no** estar presente un valor. Un **opcional** representa dos posibilidades: hay un valor y puedes usarlo; o bien, no hay ningún valor

```
struct Book {  
    var name: String  
    var publicationYear: Int  
}  
  
let firstHarryPotter = Book(name: "Harry Potter y la  
piedra filosofal", publicationYear: 1997)  
let secondHarryPotter = Book(name: "Harry Potter y la  
cámara secreta", publicationYear: 1998)  
let thirdHarryPotter = Book(name: "Harry Potter y el  
prisionero de Azkaban", publicationYear: 1999)  
  
let books = [firstHarryPotter, secondHarryPotter,  
thirdHarryPotter]
```

OPCIONALES

12.1 Nil (Nulo)

Ahora imagina que estás creando una pantalla en la que se muestra una lista de libros que han sido anunciados, pero que aún no han sido publicados. ¿Cómo podrías inicializar esos libros sin fecha de publicación? ¿Qué le asignarías a `publicationYear`?

```
let unannouncedBook1 = Book(name: "Rebeldes y leones", publicationYear: 0)

let unannouncedBook2 = Book(name: "Rebeldes y leones", publicationYear: 2023)

let unannouncedBook3 = Book(name: "Rebeldes y leones", publicationYear: nil)
```

Nil (nulo) representa la ausencia de un valor, o nada. Como todavía no hay `publicationYear`, `publicationYear` debería ser **nil**

OPCIONALES

12.1 Nil (Nulo)

Cada tipo tiene un tipo opcional correspondiente, que declaraste agregando un signo de **?** después del nombre del tipo original.

En este caso, tienes que actualizar la anotación de tipo en la propiedad `publicationYear` a **Int?**, un opcional **Int**.

```
struct Book {  
    var name: String  
    var publicationYear: Int?  
}  
  
let firstHarryPotter = Book(name: "Harry Potter y la  
piedra filosofal", publicationYear: 1997)  
let secondHarryPotter = Book(name: "Harry Potter y la  
cámara secreta", publicationYear: 1998)  
let thirdHarryPotter = Book(name: "Harry Potter y el  
prisionero de Azkaban", publicationYear: 1999)  
  
let books = [firstHarryPotter, secondHarryPotter,  
thirdHarryPotter]  
  
let unannouncedBook = Book(name: "Rebeldes y leones",  
publicationYear: nil)
```


OPCIONALES

12.2 Especificar el tipo de un opcional

Ten en cuenta que no puedes crear un **opcional** sin especificar el tipo. Piensa en lo que sucedería si intentaras que **Swift** dedujera el tipo. En este ejemplo, la inferencia de tipo asumirá que tu variable es **no opcional**:

```
var serverResponseCode = 404 // Int, no Int?
```

En este ejemplo, la inferencia de tipo no tiene ninguna información para determinar el tipo de los datos si estos no son **nil:**

```
var serverResponseCode = nil // Error, no se especificó un tipo  
cuando no es `nil`
```

OPCIONALES

12.2 Especificar el tipo de un opcional

Por estos motivos, en la mayoría de los casos tendrás que usar la **anotación de tipo** para especificar el tipo cuando crees una constante o variable opcional. Echa un vistazo a los siguientes enfoques correctos a un opcional `Int?` opcional:

```
var serverResponseCode: Int? = 404 // Establecido en 404,  
    pero podría ser `nil` más adelante
```

```
var serverResponseCode: Int? = nil // Establecido en `nil`,  
    pero podría tener un `Int` más adelante
```

OPCIONALES

12.3 Trabajar con valores opcionales

¿Cómo se determina si un **opcional** contiene o no un valor? ¿Cómo se accede al valor? Puedes empezar comparando el opcional con **nil** por medio del uso de una instrucción "if". Si el valor no es **nil**, puedes extraer el valor mediante el uso del operador **force-unwrap, !**

```
if publicationYear != nil {  
    let actualYear = publicationYear!  
    print(actualYear)  
}
```


OPCIONALES

12.3 Trabajar con valores opcionales

Si se omitió la comparación del opcional con **nil** y fuerzas la extracción de un opcional que no contiene un valor, el código generará un error y fallará cuando intentes ejecutarlo.

```
let unwrappedYear = publicationYear! // error en el tiempo  
    de ejecución  
print(unwrappedNumber)
```

”

La vinculación opcional extrae el opcional y, si contiene un valor, asigna el valor a una constante como tipo no opcional, lo que hace que sea seguro trabajar con este. Este enfoque elimina la necesidad de seguir trabajando con la ambigüedad de si se está trabajando o no con un valor o con nil

OPCIONALES

12.3 Trabajar con valores opcionales

La sintaxis para la vinculación opcional se ve así:

```
if let constantName = someOptional {  
    // constantName se extrajo de forma segura para su uso dentro  
    de las llaves  
}
```

OPCIONALES

12.3 Trabajar con valores opcionales

¿Cómo se determina si un **opcional** contiene o no un valor? ¿Cómo se accede al valor? Puedes empezar comparando el opcional con **nil** por medio del uso de una instrucción "if". Si el valor no es **nil**, puedes extraer el valor mediante el uso del operador **force-unwrap**, !

OPCIONALES

12.4 Funciones y opcionales

Si quieres denominar un valor que no cambie durante la vida del programa, deberas usar una constante.

OPCIONALES

12.5 Inicializadores falibles

Si quieres denominar un valor que no cambie durante la vida del programa, deberas usar una constante.

OPCIONALES

12.6 Encadenamiento opcional

Si quieres denominar un valor que no cambie durante la vida del programa, deberas usar una constante.

OPCIONALES

12.7 Opcionales extraídos de forma implícita

Si quieres denominar un valor que no cambie durante la vida del programa, deberas usar una constante.

13. GUARD



SWIFT

- **13.1 Guard con opcionales**

GUARD

13.1 Guard con opcionales

Si quieres denominar un valor que no cambie durante la vida del programa, deberas usar una constante.

14. PROTOCOLS



SWIFT

- **14.1 Imprimir información con CustomStringConvertible**
- **14.2 Comparar información con Equatable**
- **14.3 Creando un protocolo**
- **14.4 Delegación**

PROTOCOLS

14.1 Imprimir información con CustomStringConvertible

Si quieres denominar un valor que no cambie durante la vida del programa, deberas usar una constante.

PROTOCOLS

14.2 Comparar información con Equatable

Si quieres denominar un valor que no cambie durante la vida del programa, deberas usar una constante.

PROTOCOLS

14.3 Creando un protocolo

Si quieres denominar un valor que no cambie durante la vida del programa, deberas usar una constante.

PROTOCOLS

14.4 Delegación

Si quieres denominar un valor que no cambie durante la vida del programa, deberas usar una constante.

15. EXTENSIONS



SWIFT

- **15.1 Añadir propiedades calculadas**
- **15.2 Añadir métodos de instancia o tipo**
- **15.3 Organización de código**